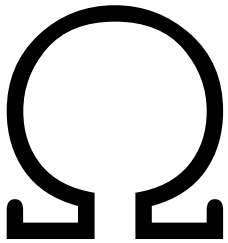

Team Omega

Audit of Backed Token

Multiplier History Update

Final Report

July 14, 2026

Summary

This document is the result of an audit by Team Omega for Backed Finance of an update of their token contact with a history of the updates of the multiplier.

We found no high severity issue - these are issues that can lead to a loss of funds, and are essential to fix. Neither did we find any "medium" issues - these are issues we believe you should definitely address. We did classify 1 issue as "low" - we believe the code would improve if these issues were addressed as well.

Severity	**Number of issues**	**Number of resolved issues**
High	0	0
Medium	0	0
Low	1	1
Info	0	0

On May 25, we submitted a preliminary report.

Subsequently, on June 16, the Backed team addressed the only issue from the preliminary report . We reviewed the fix, and noted our observations below.

Scope of the Audit

The audit concerns an upgrade of the Backed Token Contract that is developed in the following repository on github:

`https://github.com/backed-fi/backed-token-contract/`

Specifically, the scope of the audit is the difference between commit:

`a3702ad5e713ebb2b8be8f059c57eac1f613baa5`

And the following commit:

`d3de2eee7313c945e3bff1c32b568869229d488b.`

We previously audited the code up to and including this last commit before, which resulted in the following report:

`https://github.com/OmegaAudits/audits/blob/main/202605-Backed-Token-ERC4626.pdf`

Resolution

On June 16, the Backed team addressed the only issue in that report in the following commit:

`3e39bdd4596f26de4a56a2a32b6fe02719d69c2c`

We reviewed the fix, and noted our observations below. The fix was merged into the “main” branch in commit

`a76ea523d428391b59c795692097be22ec77c3ea`

Team Omega

Team Omega is a small team of auditors has many years of experience in developing smart contracts on Ethereum, and has a a pluri-annual experience auditing Solidity code

Liability

The audit is on a best-effort basis. In particular, the audit does not represent any guarantee that the software is free of bugs or other issues, nor is it an endorsement by Team Omega of any of the functionality implemented by the software.

Methods Used

The contracts were compiled, deployed, and tested in a test environment. We ran static analysis tools and AI tools.

Code Review
We manually inspected the source code to identify potential security flaws. We also inspected the context in which the contracts were developed, namely the Chainlink Bridge architecture.

Automatic analysis
We have used static analysis tools to detect common potential vulnerabilities. No high severity issues were identified with the automated processes. Some low severity issues were found, and we have included them below in the appropriate parts of the report.

Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

Severity definitions

High	Vulnerabilities that can lead to loss of assets or data manipulations.
Medium	Vulnerabilities that are essential to fix, but that do not lead to assets loss or data manipulations
Low	Issues that do not represent direct exploit, such as poor implementations, deviations from best practice, high gas costs, etc
Info	Matters of opinion

Findings

BackedAutoFeeTokenImplementation.sol

BAFT1. Unprotected v4 Migration Initializer can be front-run and permanently lock legitimate migration [low] [resolved]

The migration entrypoint `initialize_v4` is callable by anyone. The only guard is `multiplierUpdates.length == 0`. On a fresh pre-v4 proxy upgrade, this condition is true, so any account can initialize first and consume the one-time migration slot. Also, a call to `updateMultiplier` by an authorized user after the upgrade would block the call to `initialize_v4`.

An attacker can front-run migration initialization and permanently prevent the legitimate operator from executing intended migration data. This creates an integrity risk for multiplier update history (an attacker could write an inconsistent history, or the history may not contain the expected sentinel value) and an operational lockout during upgrade rollout.

Severity: Low. There is no financial risk, and no strong incentives for the attack.

Recommendation: Make sure you use `upgradeAndCall` from the proxy to initialization. The current repository contains no tests for complete upgrade procedure, nor does it contain any deployment scripts, so we cannot confirm from the repository data if you will.

Resolution: the issue was resolved as recommended, and now contains an explicit deployment script that uses the `upgradeAndCall` pattern.